

Lab 1.2 — Server Health CLI Tool

0 — Setup

Run this cell first. It installs `requests` and creates the `servers.json` input file.

```
In [1]: import subprocess, sys
subprocess.run([sys.executable, '-m', 'pip', 'install', 'requests', '-q'])

import json, pathlib

servers_data = [
    {"name": "api-prod-1", "host": "httpbin.org", "port": 443, "protocol": "https"},
    {"name": "api-prod-2", "host": "httpbin.org", "port": 443, "protocol": "https"},
    {"name": "slow-server", "host": "httpbin.org", "port": 443, "protocol": "https"},
    {"name": "unreachable", "host": "10.255.255.1", "port": 8080, "protocol": "http"}
]

pathlib.Path('servers.json').write_text(json.dumps(servers_data, indent=2))
print('✅ servers.json created with', len(servers_data), 'entries')
```

✅ servers.json created with 4 entries

Task 1 — Load the config file

Concept: `json` and `pathlib`

`pathlib.Path` is the modern way to work with files in Python. It works on all operating systems.

```
import json, pathlib

# Read a file as text, then parse JSON
raw = pathlib.Path('config.json').read_text()
data = json.loads(raw)

# Write JSON to a file
pathlib.Path('output.json').write_text(json.dumps(data, indent=2))
Always handle errors specifically:
```

- `FileNotFoundError` — the file doesn't exist
- `json.JSONDecodeError` — the file exists but isn't valid JSON

```
In [4]: import json
import pathlib
import sys
```

```
#  YOUR TURN
# Write a function load_servers(path: str) -> list[dict] that:
# 1. Reads the JSON file at the given path
# 2. Returns the parsed list of server dicts
# 3. Prints a clear error and exits if the file is not found or invalid JS

def load_servers(path: str) -> list[dict]:
    """Load and return the list of servers from a JSON file."""
    try:
        raw = pathlib.Path(path).read_text()
        return json.loads(raw)
    except FileNotFoundError:
        print(f"Error: file not found - {path}")
        sys.exit(1)
    except json.JSONDecodeError as e:
        print(f"Error: invalid JSON in {path} - {e}")
        sys.exit(1)

# Test your function
servers = load_servers('servers.json')
print(f'Loaded {len(servers)} servers')
print('First server:', servers[0])
```

Loaded 4 servers

First server: {'name': 'api-prod-1', 'host': 'httpbin.org', 'port': 443, 'protocol': 'https', 'health_path': '/status/200'}

Task 2 — Check a single server

Concept: requests + error handling

The `requests` library makes HTTP calls simple. Always set a `timeout` — otherwise your script hangs forever if a server doesn't respond.

```
import requests, time

start = time.time()
try:
    resp = requests.get('https://example.com/health', timeout=5)
    elapsed_ms = (time.time() - start) * 1000
    print(resp.status_code, elapsed_ms)
except requests.exceptions.ConnectionError:
    print('Could not connect')
except requests.exceptions.Timeout:
    print('Request timed out')
```

Status rules for our tool:

- `UP` — HTTP 200 and response time $\leq$ 500 ms
- `DEGRADED` — HTTP 200 but slow, OR a non-200 HTTP response

- **DOWN** — connection error or timeout

```
In [8]: import requests
import time

#  YOUR TURN
# Write check_server(server: dict) -> dict
# It should return a dict with keys:
#   name, url, status, response_time_ms, status_code
# For DOWN servers: response_time_ms=None, status_code=None, add an 'error'

def check_server(server: dict) -> dict:
    """Check the health of a single server and return a result dict."""
    url = f"{server['protocol']}://{server['host']}:{server['port']}"
    start = time.time()
    try:
        resp = requests.get(url, timeout=5)
        elapsed_ms = (time.time() - start) * 1000
        status_code = resp.status_code

        if status_code == 200 and elapsed_ms <= 500:
            status = "UP"
        else:
            status = "DEGRADED"

        return {
            "name": server["name"],
            "url": url,
            "status": status,
            "response_time_ms": round(elapsed_ms, 2),
            "status_code": status_code,
        }
    except requests.exceptions.ConnectionError:
        return {
            "name": server["name"],
            "url": url,
            "status": "DOWN",
            "response_time_ms": None,
            "status_code": None,
            "error": "Could not connect",
        }
    except requests.exceptions.Timeout:
        return {
            "name": server["name"],
            "url": url,
            "status": "DOWN",
            "response_time_ms": None,
            "status_code": None,
            "error": "Request timed out",
        }

# Quick test with the first (healthy) server
result = check_server(servers[0])
print(result)
```

```
{'name': 'api-prod-1', 'url': 'https://httpbin.org:443/status/200', 'status': 'DOWN', 'response_time_ms': None, 'status_code': None, 'error': 'Request timed out'}
```

Task 3 — Build the summary report

 Concept: `datetime` for timestamps

```
from datetime import datetime
now = datetime.now().isoformat(timespec='seconds')
# '2026-05-30T09:45:00'
```

In [9]: `from datetime import datetime`

```
#  YOUR TURN
# Write run_health_check(servers: list[dict]) -> dict
# It should:
# 1. Call check_server() for each server
# 2. Print a one-line status per server as it runs
# 3. Return a report dict with: checked_at, total, up, degraded, down, res

def run_health_check(servers: list[dict]) -> dict:
    """Check all servers and return a summary report."""
    results = []
    up = degraded = down = 0

    for server in servers:
        result = check_server(server)
        results.append(result)

        if result["status"] == "UP":
            up += 1
            print(f"[UP]          {result['name']} - {result['response_time_ms']}")
        elif result["status"] == "DEGRADED":
            degraded += 1
            print(f"[DEGRADED] {result['name']} - HTTP {result['status_code']}")
        else:
            down += 1
            print(f"[DOWN]      {result['name']} - {result['error']}")

    return {
        "checked_at": datetime.now().isoformat(timespec="seconds"),
        "total": len(servers),
        "up": up,
        "degraded": degraded,
        "down": down,
        "results": results,
    }

report = run_health_check(servers)
print('\nReport keys:', list(report.keys()))
```

```
[DEGRADED] api-prod-1 - HTTP 503 (542 ms)
[DEGRADED] api-prod-2 - HTTP 503 (431 ms)
[DEGRADED] slow-server - HTTP 503 (408 ms)
[DOWN]      unreachable - Could not connect
```

```
Report keys: ['checked_at', 'total', 'up', 'degraded', 'down', 'results']
```